Ejercicios para la sesión 6 de prácticas

Cosas a tener en cuenta:

- Cuando sabes de antemano cuántas iteraciones (máximas) vas a hacer es *recomendable* utilizar un bucle *for* en vez de un *while.*
- No te olvides de comentar las funciones y poner las tabulaciones de forma correcta.
- Documenta la cabecera de las funciones como has visto en clase.
- No utilices ChatGPT o hagas alguna de las malas prácticas vistas en clase de teoría ya que bajará tu nota (o incluso serás suspenso).
- El **único** elemento de consulta permitido son los apuntes del curso. No busques en internet, te copies o utilices ChatGPT sin haberlo pensado e intentado antes (varias veces!!).

Ampliación de la semana pasada: modulariza el programa para calcular MCD y MCM con una función para el menú

1. A partir del ejercicio 3 de la semana pasada sobre realizar un programa modular que pregunte al usuario si desea calcular el máximo común divisor o el mínimo común múltiplo a partir de dos números que éste introducía. En este ejercicio se pide hacer lo siguiente:
    - **Modulariza** este programa con una función *imprimeMenu*() que indique qué quiere hacer el usuario.
    - **Modulariza** también la entrada y salida del programa pasando los valores por referencia.

    Consejo: para calcular MCD/MCM usa la lógica, en lugar de algoritmos como Euclides.

Ejercicios sobre modularización y funciones

2. Realiza un programa modular que permite calcular el **área y el perímetro** de un **cuadrado**, un **círculo** o un **triángulo equilátero**. El programa debe presentar un **menú**, solicitar los **datos necesarios** según la figura seleccionada, hacer los cálculos y mostrar los **resultados de los cálculos**.

3. Escribe un programa que simule una mini aventura numérica en la que el jugador pueda **explorar distintos lugares** (bosque, castillo y pueblo), **recogiendo un objeto en cada exploración** y soltando el que ya tenía. El programa debe permitir al usuario **ir visitando lugares** mientras que no indique lo contrario. También debe **mostrar su último objeto recogido** y estar completamente modularizado mediante funciones *void* para mostrar la bienvenida, elegir el destino, recoger un objeto y mostrar el inventario. Este programa debe utilizar para esas funciones **paso por referencia para actualizar directamente la variable que representa el lugar que visitar y último objeto del jugador. Debe indicar también cuántos objetos tiene al final del juego y cuántos veces ha explorado cada lugar.**

```
==============================
  BIENVENIDO A LA AVENTURA
==============================


¿A dónde deseas ir?
1. Bosque
2. Castillo
3. Pueblo
Elige un número: 1
Exploras el bosque... encuentras una espada oxidada.

Tu inventario actual contiene: espada oxidada

¿Deseas seguir explorando? (s/n): s

¿A dónde deseas ir?
1. Bosque
2. Castillo
3. Pueblo
Elige un número: 2
Entras al castillo... hallas una corona dorada.

Tu inventario actual contiene: corona dorada

¿Deseas seguir explorando? (s/n): s

¿A dónde deseas ir?
1. Bosque
2. Castillo
3. Pueblo
Elige un número: 3
Paseas por el pueblo... consigues una bolsa de monedas.

Tu inventario actual contiene: bolsa de monedas
```

¿Deseas seguir explorando? (s/n): n

=== Fin del juego ===
Has recogido un total de 3 objetos.
Tu último objeto fue: bolsa de monedas

--- Estadísticas de exploración ---
Veces que exploraste el bosque: 1
Veces que exploraste el castillo: 1
Veces que exploraste el pueblo: 1

4. Realiza un programa que pida números enteros hasta que se introduzca el cero. El programa debe sumar todos los números pares y los números impares por separado y mostrar estas sumas. No mezcles entrada/salida con cálculos.

Recursividad

5. A partir del ejercicio 5 de la semana pasada, el cual consistía en un programa modular para calcular la potencia, factorial y combinatoria, reescribe ese programa usando recursividad.

6. Modulariza el ejercicio del *guessing game* de las sesiones anteriores utilizando **recursividad**. En este juego, el usuario piensa un número y el **programa intenta adivinarlo** realizando sucesivos intentos. Para ello, crea una función llamada *buscar* que se llame a sí misma.

Consejo: Convierte el bucle iterativo en una estructura recursiva. Llama a la función buscar mientras no se haya adivinado el número. En cada llamada, la función deberá ajustar el intervalo de búsqueda en función de las respuestas del usuario.